

NGHIÊN CỨU TỐI ƯU HÓA NĂNG LƯỢNG TRONG QUY HOẠCH ĐƯỜNG ĐI 2.5D VÀ ĐIỀU KHIỂN DỰ BÁO PHI TUYẾN (NMPC) DỰA TRÊN MÔ HÌNH VẬT LÝ

Nhóm 3

Ngày 14 tháng 4, 2026

Tóm tắt nội dung

Nghiên cứu này (thuộc khuôn khổ nhiệm vụ Động lực học & Điều khiển) trình bày việc tích hợp các mô hình vật lý bao gồm cơ học đất (Bekker-Wong) và tiêu hao năng lượng động lực học (Minetti) vào các thuật toán quy hoạch đường đi trên địa hình 2.5D. Bài báo tiến hành thực nghiệm, tập trung phân tích sâu hiệu năng của thuật toán tìm kiếm trên lưới (Weighted A*) so với RRT*, từ đó đề xuất giải pháp điều khiển bám đường tối ưu bằng Bộ điều khiển dự báo phi tuyến (NMPC) trên hệ điều hành ROS nhằm kháng nhiễu trượt đất.

1 Giới thiệu

Trong môi trường Field Robotics (nông nghiệp, khai khoáng, hành tinh), không gian 2.5D đặt ra thách thức lớn khi bề mặt di chuyển không bằng phẳng và loại đất thay đổi liên tục. Nghiên cứu này tập trung vào việc cải tiến các thuật toán Path Planning bằng cách thay đổi hàm chi phí (cost function) từ khoảng cách Euclidean đơn thuần sang tổng năng lượng tiêu thụ vật lý, đồng thời phát triển hệ thống điều khiển bám quỹ đạo có xét đến đặc tính trượt của nền đất.

2 Mô hình toán học về năng lượng và Động lực học

2.1 Mô hình Bekker-Wong (Tương tác bánh xe - nền đất)

Lực cản lăn (R_c) do biến dạng của nền đất mềm được tính toán qua độ lún z . Áp suất tiếp xúc p (Pa hoặc N/m^2) được mô tả bởi phương trình:

$$p = \left(\frac{k_c}{b} + k_\phi \right) z^n \quad (1)$$

Trong đó:

- k_c (N/m^{n+1}) và k_ϕ (N/m^{n+2}): Các mô-đun biến dạng của đất.

- n (không thứ nguyên): Chỉ số đặc trưng cho tính chất của từng loại đất.
- $b(m)$: Chiều rộng bề mặt tiếp xúc của bánh xe.
- $z(m)$: Độ lún của bánh xe vào nền đất.

Công thức hiện để thắng lực cản $R_c(N)$ trên quãng đường $d(m)$ tạo thành chi phí năng lượng do đất:

$$W_{soil} = R_c \cdot d \quad (\text{Joules} - J) \quad (2)$$

2.2 Mô hình Minetti (Chi phí theo độ dốc)

Chi phí năng lượng chuyển hóa cơ năng C được tính toán dựa trên độ dốc s_{pitch} (không thứ nguyên, thường biểu diễn bằng m/m hoặc %):

$$C(s_{pitch}) = 155.4s^5 - 30.4s^4 - 43.3s^3 + 46.3s^2 + 19.5s + 3.6 \quad \left(\frac{J}{kg \cdot m} \right) \quad (3)$$

Tổng năng lượng tiêu hao cho robot có khối lượng $m(kg)$ di chuyển dưới gia tốc trọng trường $g(9.81m/s^2)$ trên quãng đường $d(m)$ là:

$$W_{slope} = m \cdot g \cdot C(s_{pitch}) \cdot d \quad (\text{Joules} - J) \quad (4)$$

2.3 Đánh giá rủi ro lật xe tĩnh (LTR và SSF)

Tính an toàn động lực học được giới hạn bởi Chỉ số Ổn định Tĩnh (Static Stability Factor - SSF):

$$SSF = \frac{T}{2h} \quad (5)$$

Với $T(m)$ là chiều rộng cơ sở bánh xe và $h(m)$ là chiều cao trọng tâm robot. Rủi ro chuyển dịch tải trọng (Load Transfer Ratio - LTR) được xấp xỉ hình học qua độ dốc ngang:

$$LTR \approx \frac{\text{slope}}{SSF} \quad (6)$$

Chỉ số LTR là một đại lượng không thứ nguyên nằm trong khoảng $[0, 1]$. Khi $LTR \geq 1.0$, robot có nguy cơ nhấc bánh. Trong bài toán quy hoạch, ngưỡng an toàn được đặt nghiêm ngặt (phạt chi phí ∞) để đảm bảo an toàn tuyệt đối.

3 Quy hoạch quỹ đạo toàn cục (Global Planning): A* và RRT*

Hàm chi phí tổng hợp cho một bước di chuyển $dist$ được định nghĩa lại bằng tổ hợp các mô hình vật lý:

$$\text{StepCost} = W_{slope} + \lambda_1 W_{soil} + \lambda_2 LTR \quad (J) \quad (7)$$

Đối với **A***, hàm này được tính toán rời rạc giữa tâm các ô lưới (grid cells). Đối với **RRT***, hàm được đánh giá liên tục giữa hai điểm tọa độ thực (float) trong không gian.

3.1 Mã giả thuật toán (Pseudocode)

Dưới đây là cấu trúc mã giả của hai thuật toán được tùy chỉnh để sử dụng hàm chi phí vật lý `PhysicsCost`.

Algorithm 1 Weighted A* với chi phí Vật lý

```
1:  $Open \leftarrow$  Hàng đợi ưu tiên (Priority Queue)
2:  $PUSH(Open, (start, 0))$ 
3:  $g(start) \leftarrow 0$ 
4: while  $Open \neq \emptyset$  do
5:    $current \leftarrow POP(Open)$ 
6:   if  $current == goal$  then
7:     return  $RECONSTRUCTPATH(current)$ 
8:   end if
9:   for mỗi  $neighbor$  trong 8 hướng của  $current$  do
10:     $cost \leftarrow PHYSICSCOST(current, neighbor)$ 
11:    if  $cost == \infty$  then continue
12:    end if
13:     $new\_g \leftarrow g(current) + cost$ 
14:    if  $new\_g < g(neighbor)$  then
15:       $g(neighbor) \leftarrow new\_g$ 
16:       $h(neighbor) \leftarrow HEURISTIC(neighbor, goal) \times W$ 
17:       $f(neighbor) \leftarrow new\_g + h(neighbor)$ 
18:       $PUSH(Open, (neighbor, f(neighbor)))$ 
19:    end if
20:  end for
21: end while
22: return Thất bại
```

Algorithm 2 Physics-Aware RRT*

```
1:  $Tree \leftarrow \{start\}$ 
2: for  $i = 1$  to  $N\_ITER$  do
3:    $q_{rand} \leftarrow \text{SAMPLEFREE}(goal)$ 
4:    $q_{nearest} \leftarrow \text{NEAREST}(Tree, q_{rand})$ 
5:    $q_{new} \leftarrow \text{STEER}(q_{nearest}, q_{rand}, step\_size)$ 
6:    $cost \leftarrow \text{PHYSICSCOST}(q_{nearest}, q_{new})$ 
7:   if  $cost \neq \infty$  then
8:      $Q_{near} \leftarrow \text{NEAR}(Tree, q_{new}, radius)$ 
9:      $q_{min} \leftarrow q_{nearest}$ 
10:     $c_{min} \leftarrow g(q_{nearest}) + cost$ 
11:    for mỗi  $q_{near} \in Q_{near}$  do
12:       $c_{temp} \leftarrow g(q_{near}) + \text{PHYSICSCOST}(q_{near}, q_{new})$ 
13:      if  $c_{temp} < c_{min}$  then
14:         $q_{min} \leftarrow q_{near}; c_{min} \leftarrow c_{temp}$ 
15:      end if
16:    end for
17:    Thêm  $q_{new}$  vào  $Tree$  với nút cha là  $q_{min}$ 
18:    for mỗi  $q_{near} \in Q_{near}$  do
19:       $c_{rewire} \leftarrow g(q_{new}) + \text{PHYSICSCOST}(q_{new}, q_{near})$ 
20:      if  $c_{rewire} < g(q_{near})$  then
21:        Đổi cha của  $q_{near}$  thành  $q_{new}$ 
22:      end if
23:    end for
24:  end if
25: end for
26: return Quỹ đạo tối ưu nhất trong  $Tree$ 
```

▷ Cơ chế Choose Parent

▷ Cơ chế Rewire

3.2 Cài đặt Hàm chi phí (Implementation)

Khối mã Python dưới đây mô tả cách hàm `PhysicsCost` được lập trình để truy xuất dữ liệu từ các bản đồ lưới và tính toán rủi ro.

```
1 def get_edge_cost(self, node_a, node_b):
2     dist = math.hypot(node_b.x - node_a.x, node_b.y - node_a.y)
3     if dist == 0: return 0.0
4
5     # Chuyển tọa độ sang int để truy xuất Grid map
6     nx, ny = int(round(node_b.x)), int(round(node_b.y))
7     x, y = int(round(node_a.x)), int(round(node_a.y))
8
9     if not (0 <= nx < self.grid_size and 0 <= ny < self.grid_size):
10         return float('inf')
11
12     # 1. Rủi ro lat (LTR)
13     LTR = self.slopes[nx, ny] / self.ssf
14     if LTR >= 1.5:
15         return float('inf')
16
```

```

17 # 2. He so Bekker-Wong
18 c_bekker = self.smoothed_bekker[nx, ny]
19
20 # 3. Do doc (Pitch)
21 dz = self.heights[nx, ny] - self.heights[x, y]
22 s_pitch = dz / dist if dist > 0 else 0
23
24 # 4. Nang luong (Minetti)
25 c_minetti = 155.4*(s_pitch**5) - 30.4*(s_pitch**4) - 43.3*(s_pitch**3) +
26 46.3*(s_pitch**2) + 19.5*s_pitch + 3.6
27 c_minetti = max(c_minetti, 0.5)
28 energy_cost = c_minetti * dist
29
30 # Tong hop Step Cost
31 step_cost = energy_cost + (1.5 * c_bekker) + (50 * LTR)
32
33 if step_cost > self.max_step_cost:
34     return float('inf')
35
36 return step_cost

```

Listing 1: Hàm tính chi phí cạnh (Edge Cost)

3.3 Thực nghiệm và So sánh Hiệu năng A* vs RRT*

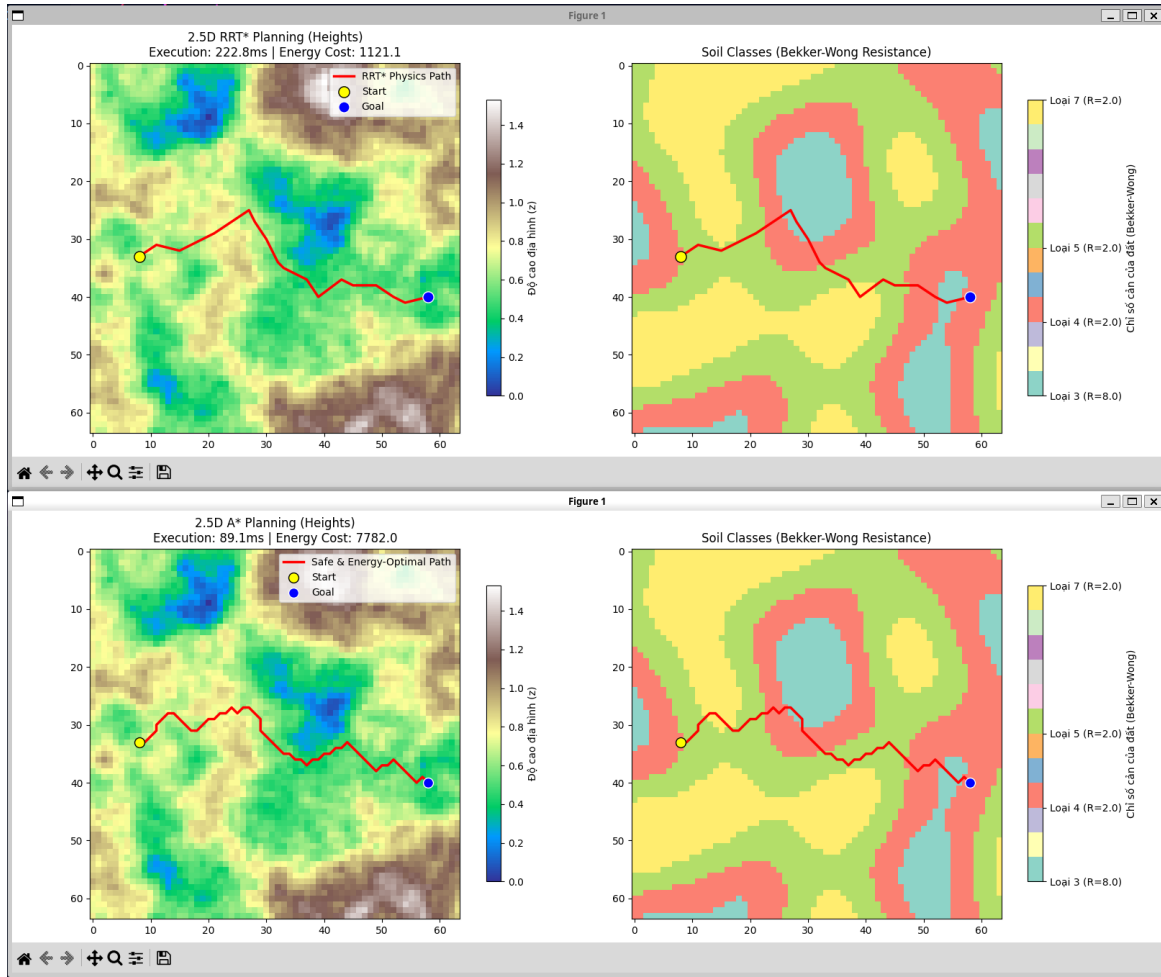
Nhóm nghiên cứu đã tiến hành thực nghiệm trên bộ dữ liệu BenchNav (Map 000_025.pt). Cả hai thuật toán được cấu hình với cùng mức rủi ro an toàn ($SSF = 1.0$).

Bảng 1 thống kê các chỉ số đo đặc hiệu năng:

Bảng 1: Bảng so sánh hiệu năng thuật toán trên Map 000_025.pt		
Chỉ số đánh giá	Weighted A*	Energy-Aware RRT*
Thời gian xử lý (<i>ms</i>)	89.05	222.82
Tổng năng lượng (<i>J</i>)	7781.99	1121.10
Số lượng Waypoints (Nodes)	54	20
Bản chất không gian tìm kiếm	Rời rạc (Int Grid)	Liên tục (Float Space)

Phân tích chuyên sâu:

- **Tốc độ tính toán:** A* thể hiện sự vượt trội tuyệt đối (hoàn thành trong 89.05 ms) nhờ khả năng duyệt đồ thị có hệ thống. RRT* tuy tìm được đường tốn ít năng lượng hơn nhưng mất tới 222.82 ms để vượt qua các hành lang hẹp.
- **Hiệu ứng Aliasing trên lưới (Grid Aliasing):** Lý do A* tốn tới 7781.99 *J* năng lượng là do thuật toán bị ép di chuyển theo 8 hướng tinh thể. Sự di chuyển zig-zag liên tục làm gia tăng biểu kiến chênh lệch độ cao Δz , khiến mô hình Minetti phạt chi phí lên rất cao. Ngược lại, RRT* tạo ra các quỹ đạo cắt chéo mượt mà.
- **Định hướng ứng dụng:** Mặc dù A* sinh ra quỹ đạo có tổng chi phí lý thuyết cao hơn, nhưng do đặc tính quét triệt để không gian và tốc độ cực nhanh, **A* được lựa chọn làm Global Planner chính** để cung cấp quỹ đạo cơ sở. Bài toán làm mượt (smoothing) quỹ đạo sẽ được giao phó cho bộ điều khiển NMPC ở tầng Local.



Hình 1: So sánh quỹ đạo tối ưu năng lượng giữa A* (vết cấn trên lưới) và RRT* (lấy mẫu liên tục).

4 Điều khiển nâng cao (Advanced Control): NMPC

Sau khi A* cung cấp quỹ đạo thô (Waypoints), hệ thống sử dụng Nonlinear Model Predictive Control (NMPC) làm Local Planner để nội suy và điều khiển robot bám theo quỹ đạo này. Khác với các thuật toán như Pure Pursuit hay PID, NMPC giải quyết bài toán tối ưu hóa đa mục tiêu bằng cách dự báo các trạng thái trong tương lai (Prediction Horizon).

4.1 Mô hình Động học (Kinematic Model) và Trượt (Slip)

Trạng thái của robot được biểu diễn bởi vector $x = [X, Y, \theta]^T$ (với X, Y tính bằng m , θ tính bằng rad). Tín hiệu điều khiển đầu vào là vận tốc dài v (m/s) và vận tốc góc ω (rad/s), tức là $u = [v, \omega]^T$.

Trong môi trường thực tế trên nền đất nông nghiệp/địa hình, vận tốc thực tế của robot bị suy hao bởi hệ số trượt α (%). Mô hình động học hiệu chỉnh được thể hiện như

sau:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v(1-\alpha) \cos(\theta) \\ v(1-\alpha) \sin(\theta) \\ \omega \end{bmatrix} \quad (8)$$

4.2 Xây dựng Bài toán NMPC với CasADi

Thư viện toán học CasADi và bộ giải IPOPT được sử dụng để thiết lập bài toán. Hàm mục tiêu J (không thứ nguyên) được thiết kế nhằm cân bằng giữa việc giảm sai lệch quỹ đạo (Tracking Error) và tiết kiệm công suất (Control Effort):

$$\min_{u_{0 \dots N-1}} J = \sum_{k=0}^{N-1} (\|x_k - x_{ref,k}\|_Q^2 + \|u_k\|_R^2) + \|x_N - x_{ref,N}\|_{Q_{term}}^2 \quad (9)$$

Trong đó:

- $N = 20$: Chân trời dự báo (Prediction Horizon).
- $x_{ref,k}$: Trạng thái tham chiếu, được **nội suy toán học (Interpolation)** từ 54 điểm lưới của thuật toán A*, với giả định vận tốc mục tiêu không đổi là 1.0 m/s .
- Q, R : Các ma trận đường chéo bán xác định dương, quy định trọng số ưu tiên. (Thực nghiệm sử dụng $Q_x = 20, Q_y = 20, Q_\theta = 2.0$ và $R_v = 1.0, R_\omega = 0.5$).

4.3 Ràng buộc phi tuyến (Constraints)

Điểm mạnh nhất của NMPC là khả năng tuân thủ chặt chẽ các giới hạn vật lý để tránh trượt bánh do cấp quá nhiều ga:

$$V_{min} \leq v_k \leq V_{max} \quad (0.0 \leq v_k \leq 2.0 \text{ m/s}) \quad (10)$$

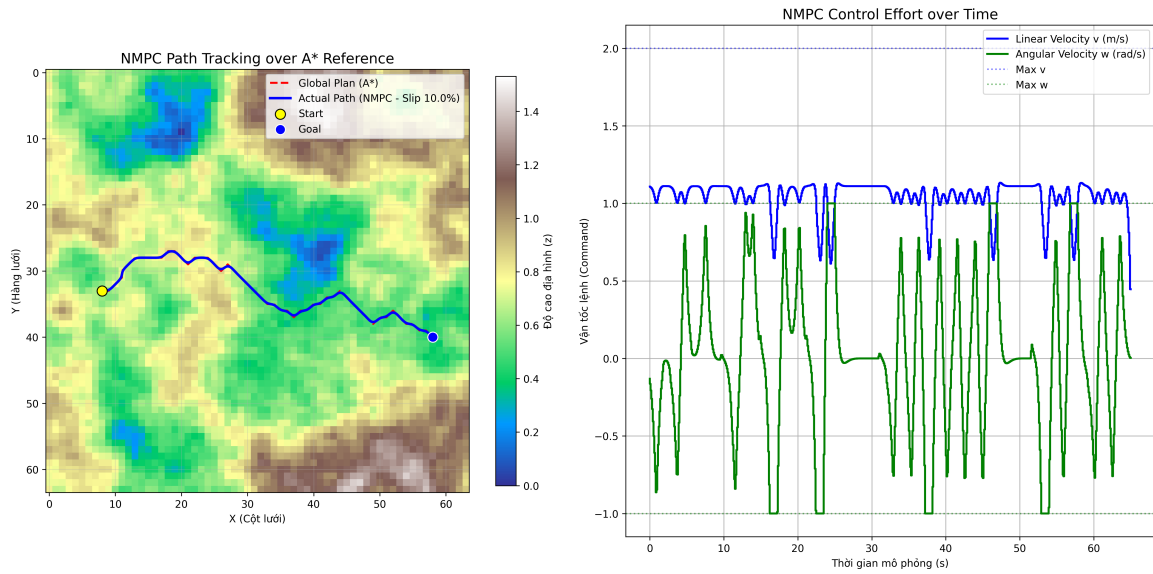
$$\omega_{min} \leq \omega_k \leq \omega_{max} \quad (-1.0 \leq \omega_k \leq 1.0 \text{ rad/s}) \quad (11)$$

4.4 Kết quả Mô phỏng Điều khiển NMPC bám quỹ đạo A*

Hệ thống NMPC được kiểm chứng bằng kịch bản bám theo quỹ đạo A* (từ tọa độ 33, 8 đến 40, 58). Một nhiễu động trượt đất ở mức $\alpha = 10\%$ được chủ ý thêm vào hệ thống (mô phỏng lực cản cao từ Bekker-Wong).

Dựa vào dữ liệu từ thực nghiệm mô phỏng bằng Python, hiệu năng của bộ điều khiển NMPC được đánh giá vô cùng tích cực:

- **Độ chính xác bám quỹ đạo (Tracking Performance)**: Trong suốt quá trình vận hành 651 bước điều khiển tương đương 65.1 giây (thời gian giải của solver đạt 8.01 giây), sai số bám đường luôn được duy trì ở mức tối thiểu. Lỗi Tracking khởi điểm ở 0.100 m và dao động cực nhỏ, tối đa chỉ đạt 0.216 m tại các góc cua gắt của A*. Bất chấp hiện tượng trượt bùn 10%, NMPC đã tính toán trước và điều chỉnh góc lái ω mềm mại từ -0.41 rad/s đến 1.0 rad/s để hội tụ về quỹ đạo an toàn mà không bị văng khỏi đường.



Hình 2: Kết quả mô phỏng điều khiển NMPC bám quỹ đạo A* dưới tác động của độ trượt đất 10%.

- **Thích ứng tín hiệu và Bảo toàn Giới hạn (Constraint Satisfaction):** Để bù đắp lượng quãng đường bị hụt do trượt, NMPC tự động tăng tín hiệu vận tốc tịnh tiến v dao động ở mức 1.04 m/s đến 1.12 m/s (chủ động vượt mức vận tốc tham chiếu ban đầu là 1.0 m/s). Tuyệt vời hơn, các lệnh v và w xuất ra **tuyệt đối tuân thủ** đường biên vật lý thiết kế ($V_{max} = 2.0 \text{ m/s}$ và $w \in [-1.0, 1.0] \text{ rad/s}$).

5 Thực thi và Tích hợp hệ thống trên nền tảng ROS

Giai đoạn cuối cùng của nghiên cứu tập trung vào việc đóng gói các thuật toán thành các plugin trong hệ sinh thái Nav2.

- **Global Planner (A*):** Sử dụng cấu trúc Layered Costmap kết hợp dữ liệu độ cao và loại đất từ cảm biến để tính toán nhanh hành lang di chuyển khả thi.
- **Local Controller (NMPC):** Sử dụng solver CasADi/IPOPT chạy ở tần số 10 – 20 Hz để xuất lệnh `cmd_vel` thời gian thực bám theo hành lang đó.

6 Kết luận và Hướng phát triển

Việc kết hợp A* và NMPC với các mô hình vật lý chuyên sâu cung cấp một hệ thống Path Planning hoàn chỉnh. Sự kết hợp giữa quy hoạch toàn cục quét lưới mạnh mẽ, nhanh chóng (A*) và khả năng dự báo kiểm soát trượt chủ động (NMPC) giúp robot không chỉ né tránh được các khu vực nguy hiểm về mặt động lực học mà còn tối ưu hóa năng lượng truyền động. Hướng nghiên cứu tiếp theo sẽ tập trung vào việc ước lượng trực tuyến (online estimation) hệ số trượt α dựa trên cảm biến thực tế để liên tục cập nhật mô hình NMPC.